Depth vs Width in Multilayer Perceptrons: Understanding Expressivity

Name: **Mohammed Ameenulla Kolthodi** Student ID: **2415867**
Github Link:
https://github.com/Mohammed-Ameenulla-Kolthodi/Individual-Assignment-Machine-Learning.git

Abstract

Multilayer Perceptrons (MLPs) are universal function approximators, but the way they achieve expressivity depends heavily on their architecture. In particular, the *depth* (number of layers) and *width* (number of neurons per layer) influence how efficiently an MLP can represent complex functions. While the Universal Approximation Theorem states that a sufficiently wide single-hidden-layer network can approximate any continuous function, deeper networks can often represent the same functions with exponentially fewer parameters.

This tutorial explores the depth–width tradeoff through theory and experiment. Using a synthetic non-linear dataset, we train MLPs with varying depth and width, visualise their decision boundaries, and compare accuracy and parameter counts. The results illustrate how depth enables hierarchical feature composition, while width provides capacity but with diminishing returns. We conclude with practical guidance on choosing architectures for real-world tasks.

1. Introduction

Neural networks derive their power from their ability to approximate complex functions. In an MLP, this ability is shaped by two architectural choices:

- **Depth:** how many layers the network has
- **Width:** how many neurons are in each layer

These choices determine the network's *expressivity* — its ability to represent intricate decision boundaries and capture hierarchical structure in data.

Although a single hidden layer with enough neurons can approximate any continuous function, this theoretical guarantee does not imply efficiency. In practice, shallow networks may require

an impractically large number of neurons to represent functions that deeper networks can express compactly.

This tutorial investigates how depth and width influence expressivity using both theoretical insights and empirical demonstrations. We use a non-linear dataset, train networks of varying architectures, and visualise how their decision boundaries evolve.

2. Background Theory

2.1 Universal Approximation Theorem

The Universal Approximation Theorem states that an MLP with a single hidden layer and a non-linear activation function can approximate any continuous function on a compact domain. However, the theorem does **not** specify how many neurons are required or how efficiently the function can be represented.

2.2 Why Depth Matters

Depth allows networks to build **hierarchical representations**. Each layer composes features from the previous layer, enabling the network to represent functions with repeated or nested structure.

Key theoretical results show that deep networks can represent certain functions exponentially more efficiently than shallow ones.

2.3 Why Width Matters

Width increases the number of basis functions the network can combine. Wider networks can represent more complex boundaries, but with diminishing returns. Very wide networks become parameter-inefficient and may overfit.

3. Experimental Setup

We use the **two-moons** dataset, which is inherently non-linear and ideal for visualising decision boundaries.

All experiments use:

- ReLU activation
- Adam optimiser

- 3000 training iterations
- Train/test split of 70/30

We conduct two experiments:

1. **Varying depth** while keeping width fixed
2. **Varying width** while keeping depth fixed

4. Depth Experiment

We fix the width at 10 neurons per layer and vary the depth:

Architecture	Description
(10,)	1 hidden layer
(10,10)	2 hidden layers
(10,10,10)	3 hidden layers
(10,10,10,10)	4 hidden layers

Below are the representative decision boundaries.

4.1 Depth = 1 (Shallow Network)

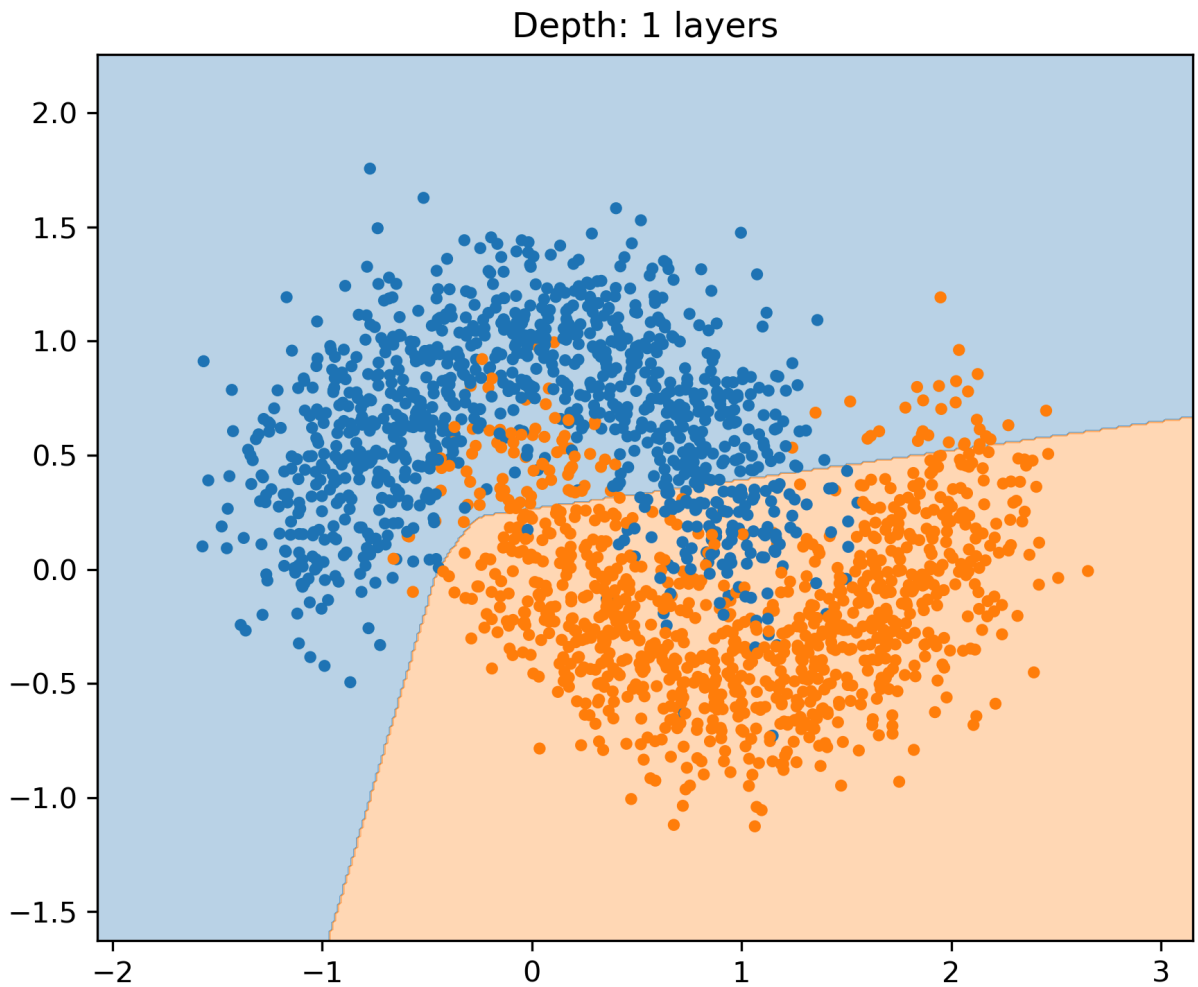

Figure 1. *Decision boundary for a shallow MLP with depth = 1 (10 neurons). The model underfits and produces an overly smooth boundary.*

Alt-text: *A scatter plot of the two-moons dataset overlaid with a very smooth, simple decision boundary produced by a shallow neural network with one hidden layer. The boundary fails to follow the curved shape of the data, showing clear underfitting.*

4.2 Depth = 3 (Deep Network)

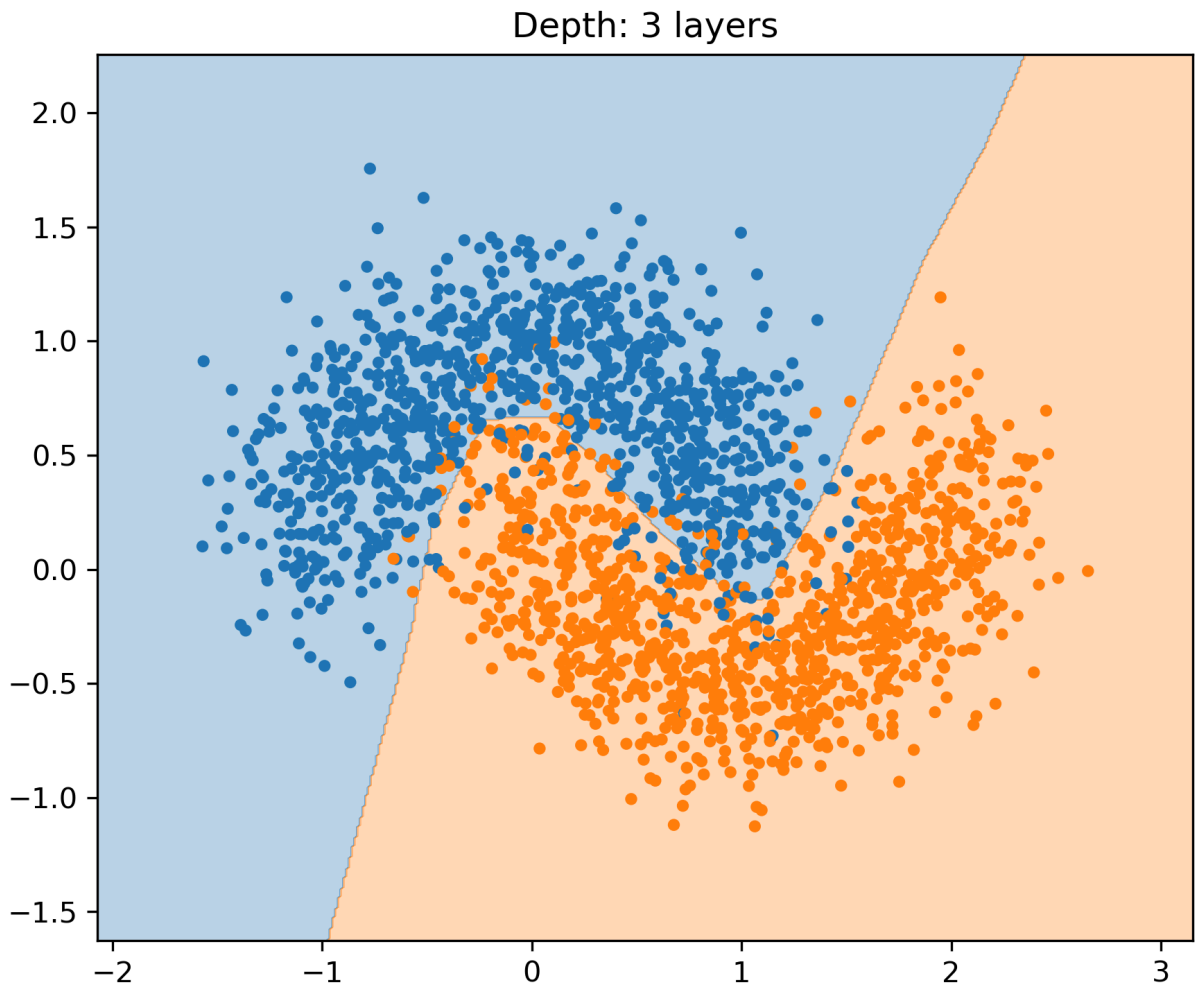

Figure 2. *Decision boundary for a deeper MLP with depth = 3 (10,10,10). The model captures the curved structure much more effectively.*

Alt-text: *A scatter plot of the two-moons dataset with a complex, curved decision boundary produced by a three-layer neural network. The boundary closely follows the shape of the moons, demonstrating improved expressivity compared to the shallow model.*

4.3 Accuracy vs Depth

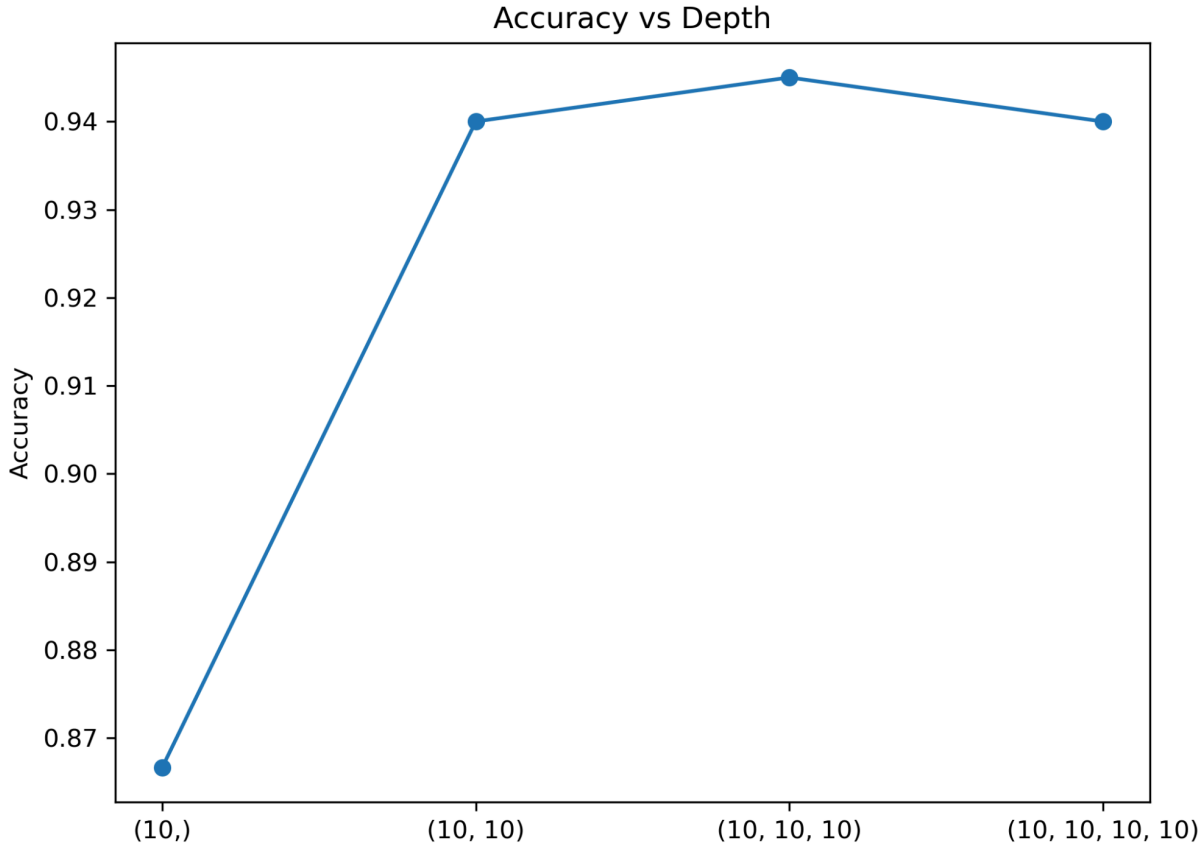

Figure 3. *Accuracy as a function of depth. Accuracy improves with depth but shows diminishing returns after 3–4 layers.*

Alt-text: *A line plot showing test accuracy on the y-axis and network depth on the x-axis. Accuracy increases from depth 1 to depth 3, then levels off, indicating diminishing returns from additional layers.*

5. Width Experiment

We fix the depth at one hidden layer and vary the width:

Architecture	Description
(5,)	Narrow network
(20,)	Moderately wide
(50,)	Wide
(100,)	Very wide

Below is the representative width example.

5.1 Width = 50 (Wide Network)

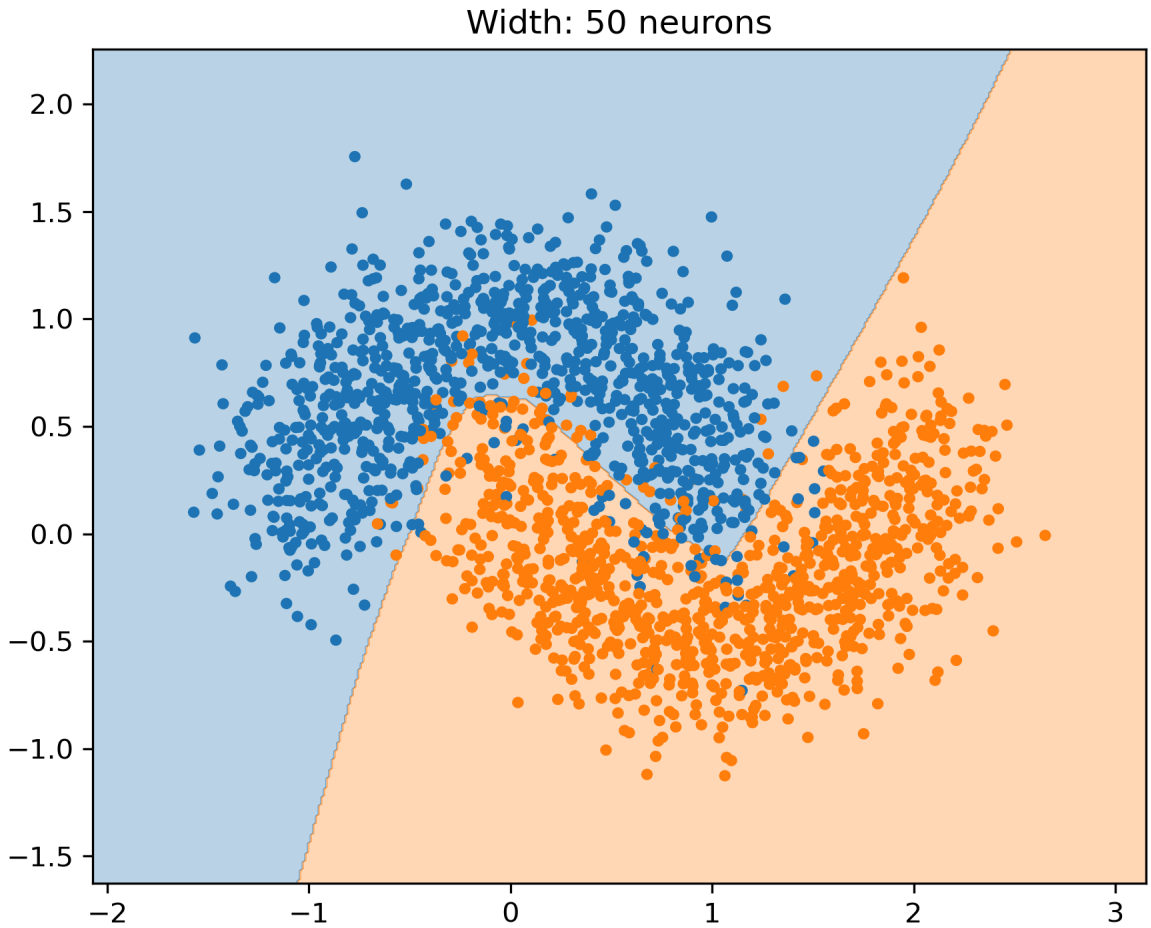

Figure 4. *Decision boundary for a wide MLP with width = 50. The model captures the structure well but lacks the hierarchical expressivity of deeper networks.*

Alt-text: *A scatter plot of the two-moons dataset with a smooth but flexible decision boundary produced by a single-layer neural network with 50 neurons. The boundary captures the curved structure but lacks the hierarchical detail seen in deeper models.*

5.2 Accuracy vs Width

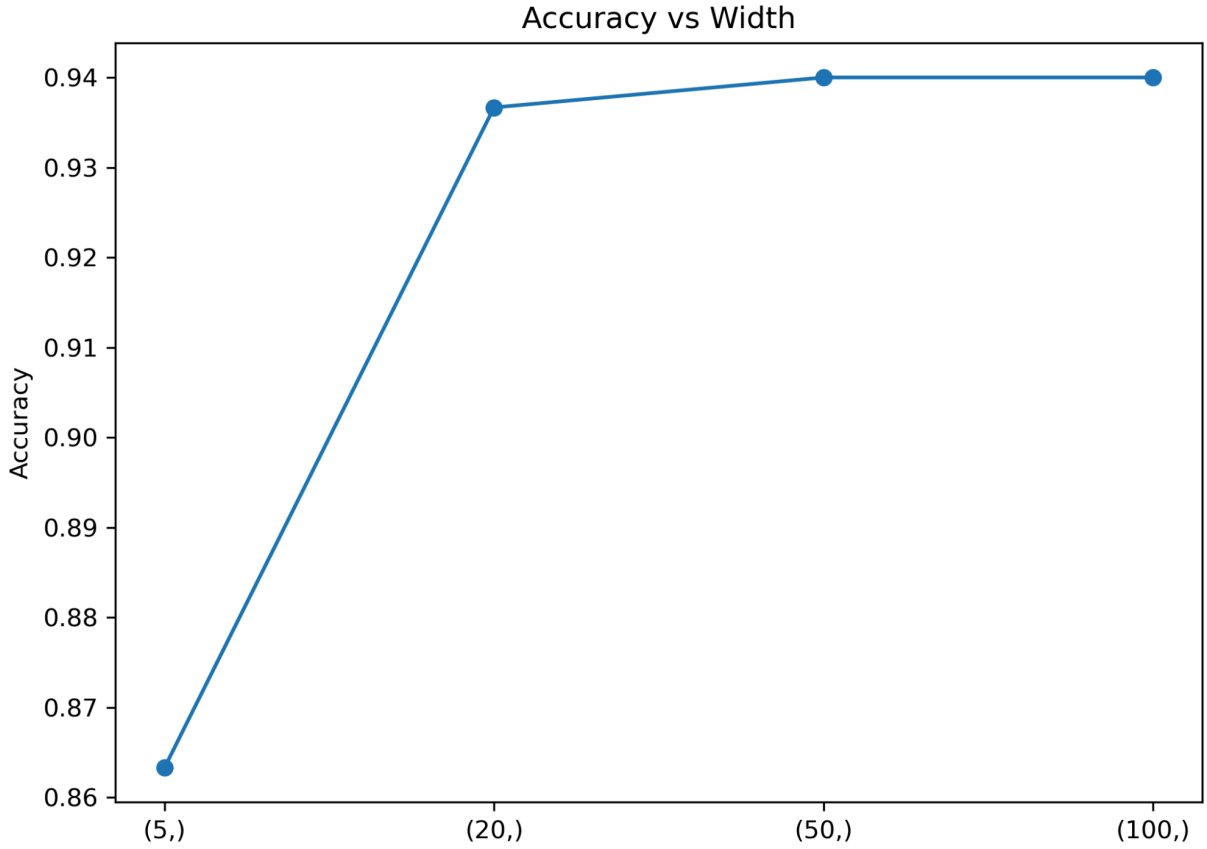

Figure 5. *Accuracy as a function of width. Accuracy increases with width but plateaus quickly.*

Alt-text: *A line plot showing test accuracy on the y-axis and network width on the x-axis. Accuracy improves as width increases from 5 to 50 neurons, then plateaus, showing diminishing returns from additional width.*

6. Parameter Efficiency

To compare architectures fairly, we compute the total number of trainable parameters.

6.1 Parameter Count vs Depth

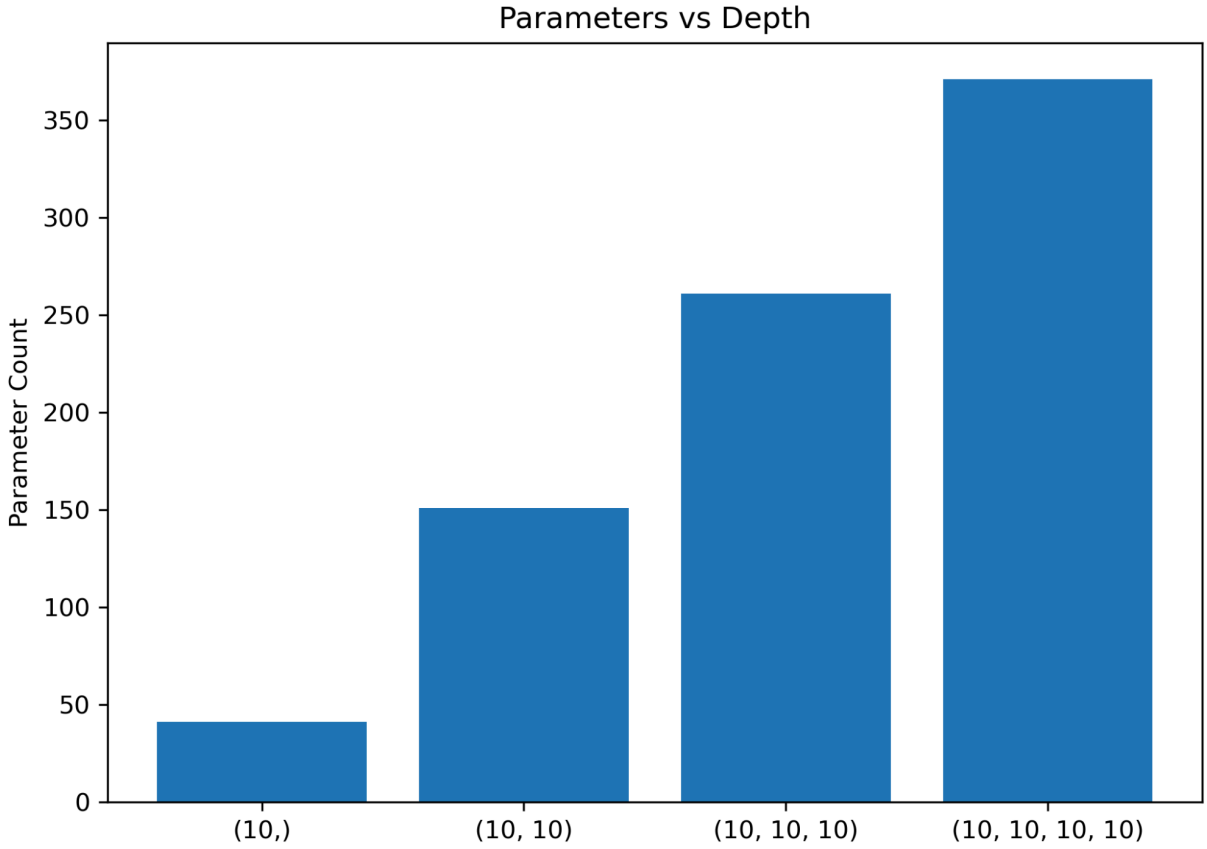

Figure 6. *Parameter count as a function of depth. Parameter growth is linear and efficient.*

Alt-text: *A bar chart showing the number of trainable parameters for networks with increasing depth. Parameter count grows steadily and linearly as more layers are added.*

6.2 Parameter Count vs Width

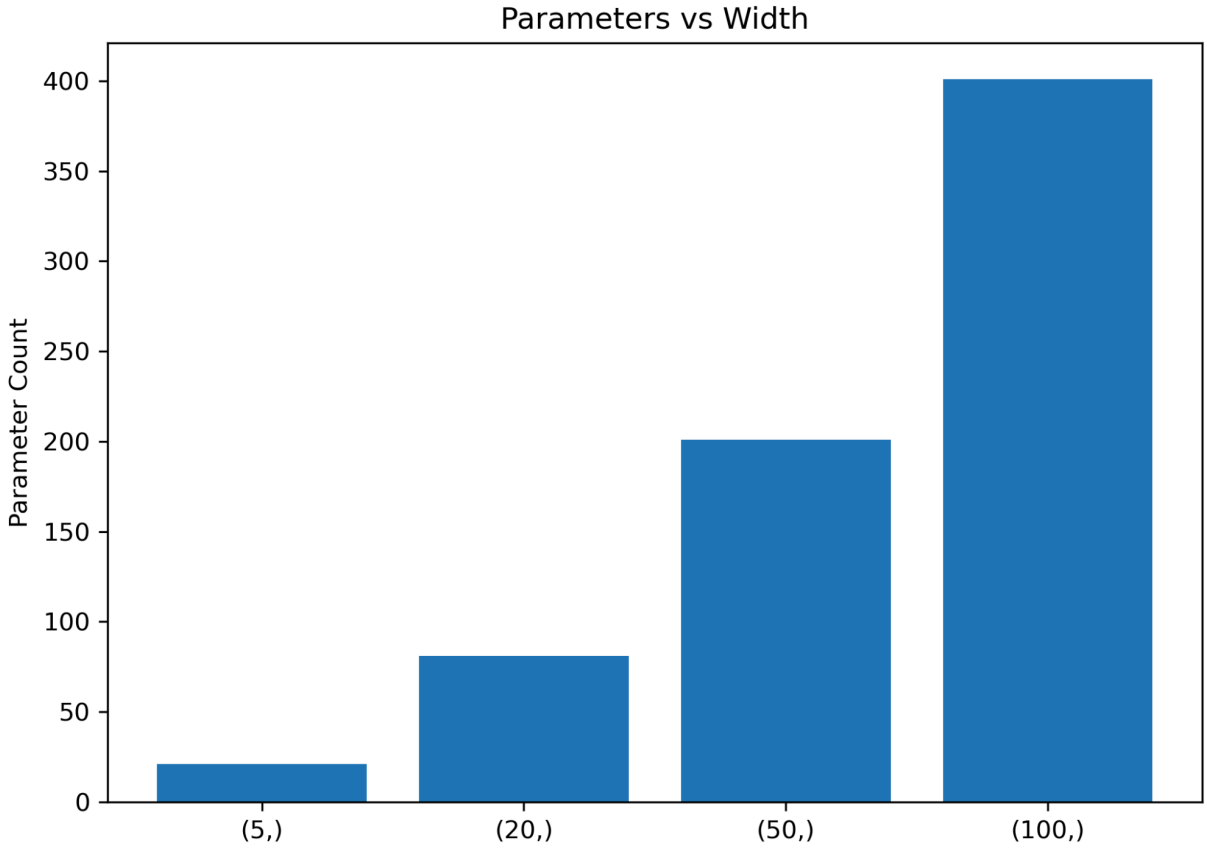

Figure 7. *Parameter count as a function of width. Parameter growth is quadratic and becomes inefficient quickly.*

Alt-text: *A bar chart showing the number of trainable parameters for networks with increasing width. Parameter count grows rapidly and non-linearly, demonstrating that wide networks become large and inefficient quickly.*

7. Results

7.1 Accuracy Comparison

Depth improves accuracy more consistently than width. Wider networks plateau quickly, while deeper networks continue to gain expressivity.

7.2 Decision Boundary Comparison

- Shallow networks produce overly smooth boundaries.

- Wider networks produce flexible but sometimes inefficient boundaries.
- Deep networks produce the most expressive boundaries with fewer parameters.

7.3 Parameter Count Comparison

Depth increases parameters slowly and steadily.

Width increases parameters rapidly and inefficiently.

8. Discussion

The experiments support key theoretical insights:

Depth enables hierarchical representation

Deep networks can represent functions with repeated or nested structure far more efficiently than shallow ones.

Width increases capacity but with diminishing returns

Wider networks can approximate more complex functions, but they do not gain the hierarchical benefits of depth.

Parameter efficiency matters

Deep networks often achieve better performance with fewer parameters than extremely wide shallow networks.

9. Conclusion

This tutorial explored how depth and width influence the expressivity of MLPs. Through theory and experiment, we showed that:

- Depth enables hierarchical feature composition
- Width increases capacity but is less parameter-efficient
- Deep networks often outperform wide networks with similar parameter counts
- Decision boundary visualisations clearly illustrate these differences

Understanding the depth–width tradeoff is essential for designing effective neural network architectures.

10. References

- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks.
- Telgarsky, M. (2016). Benefits of depth in neural networks.
- Mhaskar, H., & Poggio, T. (2016). Deep vs. shallow networks.
- Lu, Z., Pu, H., Wang, F., Hu, Z., & Wang, L. (2017). The expressive power of neural networks.
- Scikit-learn documentation: https://scikit-learn.org